

Write path — one drawer, and why it is safe

Content is stored verbatim. Only derived structure is added; the text itself is never rewritten.

1 • normalize

safety floor always: strip NUL and control chars, CRLF → LF

Code mode keeps every space

2 • derive, never alter

time_mentions · entities
content_date anchors relative dates
extracted in new(); anchored if dated

3 • embed

deterministic offline hash embedder
by default — no model, no network
computed before the lock is taken

4 • at the write choke point: validate the declaration, then screen

wing/room/kind names, the id shape and a finite vector are checked first, and refused as invalid input — never rewritten by the store.

Then the Screen every write_drawer must state: Apply (tier-1 signals · declared rate · advisor) or Bypass(reason). A flagged write is DIVERTED into the review wing and s

5 • one SQLite transaction — all of it, or none of it

BEGIN IMMEDIATE

seal

XChaCha20-Poly1305 over content
and embedding.

AAD = the drawer id, plus distinct
domains /emb · /tok · /pq

tag

HMAC-SHA256 over
id || meta_json || sealed content
metadata is covered for free — a new
meta field needs no schema change

chain

chain_append runs in the SAME
transaction as the row.

so no committed write can exist
without its audit entry

Bulk ingest: one transaction and one anchor per batch, not per drawer — measured ≈55× fewer fsyncs at 200 drawers.

6 • then, outside the transaction

fsync the manifest anchor through an atomic
rename + directory sync. WAL, synchronous=FULL.

why the anchor lags on purpose

anchor behind the db at open → power loss, fast-forward
anchor ahead of the db at open → rollback, tamper alarm

The asymmetry is the detector

An anchor that never runs ahead of the database is what makes "the machine lost power" and "someone restored an older copy of this file" distinguishable at all. That is why the invariant is directional, not just "keep them in sync".